Client Server and Peer-to-Peer Models

A client server and a Peer-to-Peer Models are two different architectures for designing networks and distributed systems.

Even though they function differently, they can actually work together in specifics enviroments.

Client Server model:
In this architecture, multiple clients (devices or software) request and receive services from a centralized server

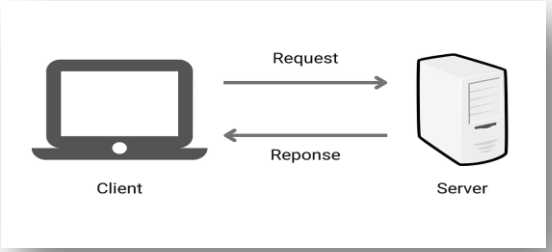

Image credit: enjoyalgorithms.com

Peer-to-Peer model:
In this architecture, each node,
or *"peer",* is directly enabled to share resources among peers without the need for centralized coordination. It is often used for file sharing, communications, and collaborative tasks.

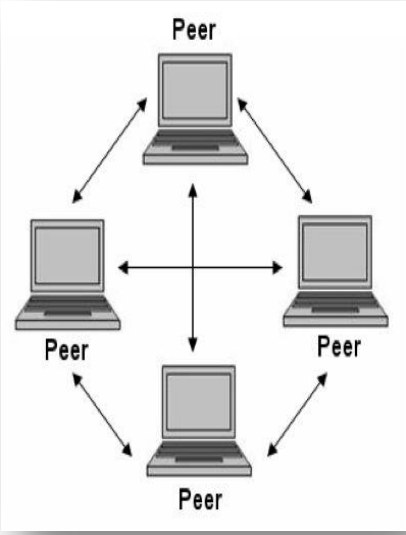

Image credit: definicionabc.com

Difference between these two architectures

As we see from the images above

In the Client Server Model:

The Client:

- Requests services or resources

The Server:

- Provides resources and services.
- Manages all processes and data storage.
- Sends output back to the client

In the Peer-to-Peer Model

Each peer can be both a server and a client:

- Requests services or resources.
- Provides resources and services.
- Manages processes and data storage.
- Sends output back to the others.

Advantages and disadavantages

Obviously these architectures have several advantages and disadavantages:

Client Server Architecture :

Advantages:

1. **Centralized Management**
2. **Security**
3. **Reliability**
4. **Scalability**
5. **Easier Backup & Data Management**

Disadvantages:

1. **Single Point of Failure**
2. **Cost**
3. **Network Traffic**
4. **Dependence on Server**

Peer-to-Peer Architecture:

Advantages:

1. **Decentralized**
2. **Cost-Effective**
3. **Scalability**
4. **Redundancy**
5. **Efficiency**

Disadvantages:

1. **Security**
2. **Limited Control**
3. **Network Congestion**
4. **Complexity in Management**

In summary:

• Client-Server is best for environments that require **centralized control**, **security**, and **reliable performance**, but it can suffer from a **single point of failure** and high costs.

• Peer-to-Peer is more **cost-effective**, **scalable**, and **resilient**, but it can be less secure and harder to manage due to its **decentralized** nature.

Implementations Enviroments

These architectures have revolutionized communication between computers and the way we interact with digital services nowadays.

For example the famous app and web platform Spotify uses the client-server model.

Spotify History:
Spotify was founded in 2006 in Stockholm, Sweden, by Daniel Ek and Martin Lorentzon and was released in 2008. The idea behind this platform are to curb online piracy, the accessibility to listen music without additional costs to download music files and, unlike the traditional music industry the artists are paid based on every song sold.

The Spotify System Model

To communicate through the software and use the catalog, Spotify presented its own System Model based on various entities and abstraction, the three main ones are:

1. **APIs:** the boundaries set between multiple components of the infrasctuture that define the interface between them
2. **Components:** Certain instances of a given software (for example, a website, data pipeline, library, etc.).
3. **Resources**: The backend infrastructure behind each component (database, storage bucket, etc.)

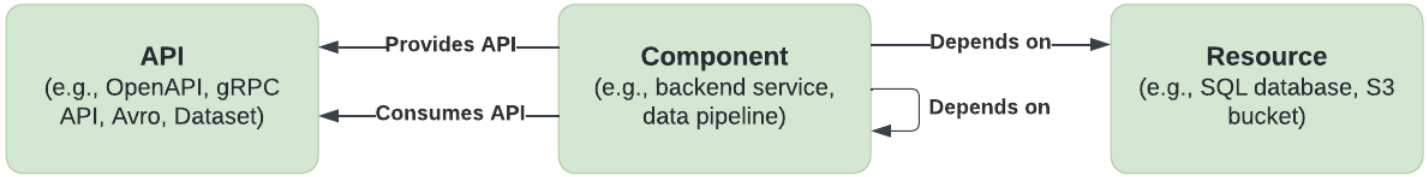

Image credit: engineering.atspotify.com

Also Spotify is based entirely on **Microservices architecture** that guarantees scalability and flexibility to enabling multiple services on differents platforms and devices.

Understanding Microservice Architecture on Spotify

This kind of architecture allows to split the heavy parts of the application into smaller parts, each one being indipendant from the others and delivering its own functionality.

The devices and the digital platforms interact with Spotify through APIs (Application Programming Interfaces) that allow access to functionalities, such as music playback or playlist synchronization.

Many microservices are developed simultaneously to solve users' problems independently and internally communicate with other services.

They are largely used to develop and deploy heavy applications.

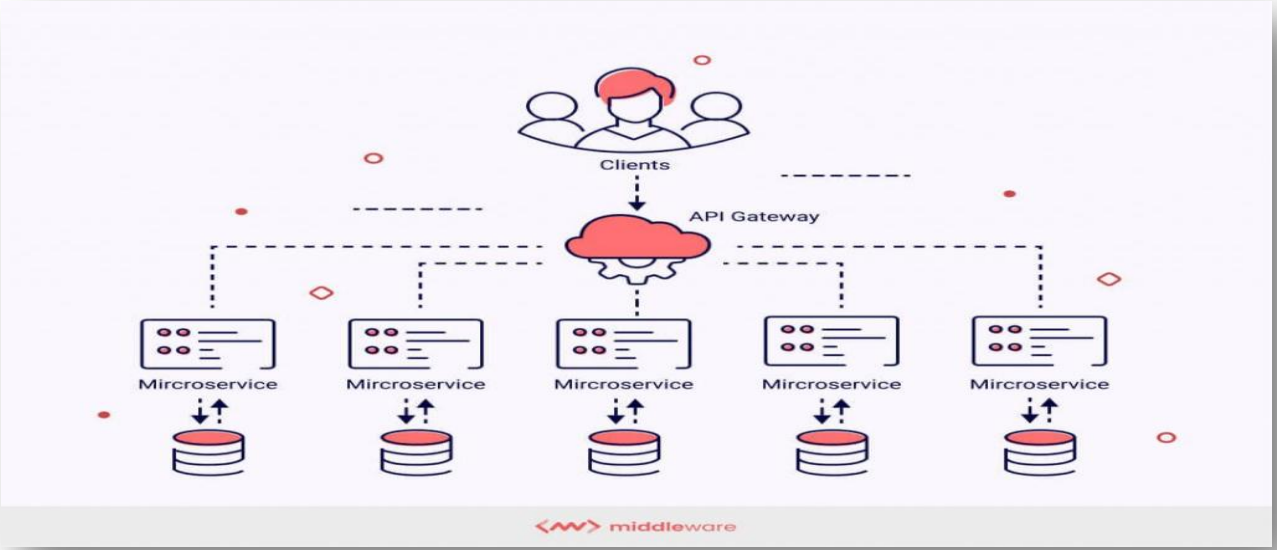

Image credit: middleware.io

Microservice architecture in Spotify

The backend of Spotify needs hundreds of services, most tiny and straightforward, but they must be served in run-time without delay. To provide these hundreds of services, Spotify's technical team deployed a microservices-based architecture.

Services are usually written in Python and/or Java, with few exceptions.

A highly specialized backend service called 'access point' is constantly in touch with various clients for swift and precise communication. The protocol, which enabled seamless communication between the clients and the 'access point,' has been built by Spotify engineers and hasn't been made public. Spotify's Software infrastructure heavily depends on Debian, while open-source software is preferred for various activities.

References

- K. Varshneya, 2022, ***Decoding Software Architecture Of Spotify: How Microservices Empowers Spotify***.Teachaheadcorp .https://www.techaheadcorp.com/blog/decoding-software-architecture-of-spotify-how-microservices-empowers-spotify/
- R. Kalman et. J. Wallin, 2022, **Software Visualization — Challenge, Accepted**. Spotify Engineering. https://engineering.atspotify.com/2022/07/software-visualization-challenge-accepted/
- Spotify Engineering, 2013, **Backend infrastructure at Spotify.** Spotify Engineering. https://engineering.atspotify.com/2013/03/backend-infrastructure-at-spotify/